

PYTHON CODING ASSESSMENT

What is Data Cleaning?

Data Cleaning (or **Data Cleansing**) is the process of:

- Detecting and correcting (or removing) **inaccurate, incomplete, duplicate, or irrelevant** parts of the data.
- Making raw data **consistent, reliable, and ready for analysis** or machine learning.

Why is Data Cleaning Important?

- Dirty data leads to **wrong results** in data analysis.
- Clean data ensures **better decisions, accurate models, and more insights**.
- It's a critical **preprocessing step** in any Data Science or Analytics project.

What Are Common Data Issues?

1. **Missing values** (NaN, blank cells)
2. **Duplicate records**
3. **Incorrect or invalid entries** (e.g., age = -5)
4. **Wrong data types** (e.g., string instead of integer)
5. **Inconsistent formats** (e.g., date in different formats)

What Did We Do in the Program?

We created a **simple Python program using Pandas** to clean a sample dataset in three ways:

1. Removing Null Values

- Used `dropna()` to remove rows where any column has a NaN value.

2. Removing Duplicates

- Used `drop_duplicates()` to keep only unique rows.

3. Fixing Incorrect Values

- Replaced invalid data (e.g., negative age) with a calculated mean using `apply()`.

Install Pandas and Import, Let's create a dummy dataset for cleaning:

```
1  import pandas as pd
2
3  # Raw data with issues: nulls, duplicates, incorrect values
4  data = {
5      'id': [1, 2, 3, 4, 4, 5, 6, None],
6      'name': ['Alice', 'Bob', 'Charlie', 'David', 'David', 'Eva', 'Frank', 'George'],
7      'age': [25, 30, None, 45, 45, -1, 33, 40],
8  }
9
10 df = pd.DataFrame(data)
11
12 print("Original Data:")
13 print(df)
```

Original Data:

	id	name	age
0	1.0	Alice	25.0
1	2.0	Bob	30.0
2	3.0	Charlie	NaN
3	4.0	David	45.0
4	4.0	David	45.0
5	5.0	Eva	-1.0
6	6.0	Frank	33.0
7	NaN	George	40.0

Program 1: Remove Null Values

```
# Remove rows with any null values
df_no_nulls = df.dropna()

print("\n After Removing Null Values:")
print(df_no_nulls)
```

After Removing Null Values:

	id	name	age
0	1.0	Alice	25.0
1	2.0	Bob	30.0
3	4.0	David	45.0
4	4.0	David	45.0
5	5.0	Eva	-1.0
6	6.0	Frank	33.0

Function Used:

- `dropna()` – removes rows with missing (NaN) values

Program 2: Remove Duplicates

```
# Remove duplicate rows from null-free data
df_no_duplicates = df_no_nulls.drop_duplicates()

print("\n After Removing Duplicates:")
print(df_no_duplicates)
```

```
After Removing Duplicates:
   id  name  age
0  1.0  Alice  25.0
1  2.0   Bob  30.0
3  4.0  David  45.0
5  5.0   Eva  -1.0
6  6.0  Frank  33.0
```

Function Used:

- `drop_duplicates()` – removes duplicate rows

Program 3: Replace Incorrect Values

```
# Replace negative age values with mean of valid (positive) ages
valid_age_mean = df_no_duplicates[df_no_duplicates['age'] > 0]['age'].mean()

# Replace -1 or any age < 0 with mean
df_no_duplicates['age'] = df_no_duplicates['age'].apply(
    lambda x: valid_age_mean if x < 0 else x
)

print("\n After Replacing Invalid Age Values (like -1):")
print(df_no_duplicates)
```

After Replacing Invalid Age Values (like -1):

	id	name	age
0	1.0	Alice	25.00
1	2.0	Bob	30.00
3	4.0	David	45.00
5	5.0	Eva	33.25
6	6.0	Frank	33.00

Functions Used:

- `apply()` – applies a function to each element
- `lambda` – anonymous function
- `mean()` – to get average

Summary of Functions Used

Function	Purpose
<code>dropna()</code>	Removes rows with any missing (NaN) values
<code>drop_duplicates()</code>	Removes duplicate rows
<code>mean()</code>	Calculates average of valid (non-null and positive) values
<code>apply() + lambda</code>	Replaces values based on conditions (e.g., negative values)

Pandas

Pandas is a popular open-source Python library used for **data manipulation, cleaning,** and **analysis**. It provides fast and flexible data structures like:

- **Series** – one-dimensional labelled array.
- **DataFrame** – two-dimensional labelled data structure, like a spreadsheet or SQL table.

Pandas makes it easy to load, clean, filter, transform, analyze, and visualize data.

Types of Joins in Pandas

Joins are used to combine rows from **two DataFrames** based on a common column (like in SQL).

Join Type	Description
Inner Join	Returns only the rows with matching keys in both DataFrames.
Left Join	Returns all rows from the left DataFrame, and matched rows from the right one.
Right Join	Returns all rows from the right DataFrame, and matched rows from the left one.
Outer Join	Returns all rows from both DataFrames. Fills missing matches with NaN.

Step 1: Import pandas and create DataFrames

```
import pandas as pd

# Customer data
customers = pd.DataFrame({
    'customer_id': [1, 2, 3, 4],
    'customer_name': ['Alice', 'Bob', 'Charlie', 'David']
})

# Order data
orders = pd.DataFrame({
    'order_id': [101, 102, 103, 104],
    'customer_id': [2, 4, 4, 5], # customer_id 5 does not exist in customers
    'product': ['Laptop', 'Phone', 'Tablet', 'Camera']
})

print(" ♦ Customers Table:")
print(customers)

print("\n ♦ Orders Table:")
print(orders)
```

```
 ♦ Customers Table:
   customer_id customer_name
0             1         Alice
1             2           Bob
2             3        Charlie
3             4         David

 ♦ Orders Table:
   order_id  customer_id product
0        101            2  Laptop
1        102            4   Phone
2        103            4  Tablet
3        104            5  Camera
```

Step 2: Perform an **INNER JOIN**

```
# Inner join: Only customers who have placed orders
inner_join = pd.merge(customers, orders, on='customer_id', how='inner')

print("\n✅ INNER JOIN (Only matching customer_id in both):")
print(inner_join)
```

```
✅ INNER JOIN (Only matching customer_id in both):
  customer_id customer_name  order_id product
0           2           Bob         101  Laptop
1           4          David         102   Phone
2           4          David         103  Tablet
```

Step 3: Perform a **LEFT JOIN**

```
# Left join: All customers, matched orders if available
left_join = pd.merge(customers, orders, on='customer_id', how='left')

print("\n✅ LEFT JOIN (All customers, matching orders if any):")
print(left_join)
```

```
✅ LEFT JOIN (All customers, matching orders if any):
  customer_id customer_name  order_id product
0           1          Alice         NaN     NaN
1           2           Bob        101.0  Laptop
2           3          Charlie         NaN     NaN
3           4          David        102.0   Phone
4           4          David        103.0  Tablet
```


Step 4: Perform a **RIGHT JOIN**

```
# Right join: All orders, matched customers if available
right_join = pd.merge(customers, orders, on='customer_id', how='right')

print("\n✅ RIGHT JOIN (All orders, matching customer info if any):")
print(right_join)
```

```
✅ RIGHT JOIN (All orders, matching customer info if any):
  customer_id customer_name  order_id product
0           2           Bob         101  Laptop
1           4          David         102   Phone
2           4          David         103  Tablet
3           5           NaN         104  Camera
```

Step 5: Perform an **OUTER JOIN**

```
# Outer join: All customers and all orders, match if possible
outer_join = pd.merge(customers, orders, on='customer_id', how='outer')

print("\n✅ OUTER JOIN (All records from both tables):")
print(outer_join)
```

```
✅ OUTER JOIN (All records from both tables):
  customer_id customer_name  order_id product
0           1          Alice         NaN     NaN
1           2           Bob        101.0  Laptop
2           3         Charlie         NaN     NaN
3           4          David        102.0   Phone
4           4          David        103.0  Tablet
5           5           NaN        104.0  Camera
```